

Weather Station

Software per servizi metereologici

Giulio Nencini - MAT: 7115972

Yuri Bartoletti - MAT: 7113273

25 giugno 2026

Indice

1	Presentazione del progetto	3
1.1	Statement	3
1.2	Pratiche e ambienti di sviluppo	3
2	Progettazione	4
2.1	Use Case Diagram	4
2.2	Use Case Template	6
2.3	Class Diagram	10
2.3.1	Domain Model	10
2.3.2	Business Logic	10
2.3.3	ORM	11
2.4	Relationship Model	12
2.5	Navigation Diagram	13
3	Implementazione	13
3.1	Domain Model	13
3.1.1	SystemUser	13
3.1.2	Notification	14
3.1.3	Measurement	14
3.1.4	Ticket	14
3.1.5	Sensor	14
3.1.6	Enumerazioni	14
3.2	Business Logic	14
3.2.1	AdminDatabaseController	14
3.2.2	AdminController	15
3.2.3	MaintainerController	15
3.2.4	SystemMonitor	15
3.2.5	SensorManager – D. P. Observer	15
3.2.6	SensorMonitor – D. P. Observer	15
3.2.7	UserController	15
3.2.8	StaffLoginController	16
3.2.9	UserLoginController	16
3.3	ORM	16
3.3.1	DatabaseManager e interfaccia AutoClosable	16
3.3.2	AdminDAO e MaintainerDAO	16
3.3.3	AlertRuleDAO	16
3.3.4	MeasurementDAO	16
3.3.5	NotificationDAO	16
3.3.6	SensorDAO	16

3.3.7	TicketDAO	17
3.3.8	UserDAO	17
4	Testing	17
4.1	Business Logic Test	18
4.1.1	AdminControllerTest	18
4.1.2	MaintainerControllerTest	19
4.1.3	UserLoginControllerTest	21
4.1.4	UserControllerTest	22
4.2	Domain Model	22
4.2.1	AlertRuleTest	22

1 Presentazione del progetto

1.1 Statement

Il software sviluppato è un sistema composto principalmente da due parti: misurazione di alcune grandezze fisiche meteorologiche ad opera di alcuni sensori, tali misurazioni saranno visibili dagli utenti che desidereranno vederle, e sistema di notificazione agli utenti iscritti qualora una di queste misurazioni dovesse violare dei limiti impostati nelle "AlertRule" dall'utente stesso. Vi è inoltre un sistema interno che si occupa della sorveglianza dei sensori: qualora si dovessero riscontrare misurazioni anomale scatterà l'apertura di un ticket e ci sarà un manutentore che provvederà alla riparazione o alla sostituzione del sensore guasto.

1.2 Pratiche e ambienti di sviluppo

Il software è stato interamente sviluppato in Java, mentre per il database è stato usato PostgreSQL mediante la libreria JDBC. Tutti i diagrammi sono stati realizzati con Gaphor, tranne il modello relazionale che è stato realizzato mediante il sito DrawSQL.app e il navigation diagram che è stato realizzato con draw.io. Per quanto riguarda la progettazione, abbiamo optato per la separazione del software in tre diversi package:

- **Business Logic:** per le classi che implementano la logica di business del sistema.
- **Domain Model:** contiene le classi che rappresentano le entità del sistema.
- **ORM:** contiene le classi che implementano l'Object-Relational Mapping, fondamentale al fine di rendere i dati permanenti in un db.

Per poter interagire col software è stata creata un'interfaccia a riga di comando.

Il codice è presente al seguente repo di GitHub: https://github.com/Yuri0303/Weather_Station_SWE

Durante lo sviluppo di questo progetto è stato fatto uso di intelligenza artificiale generativa per debug, suggerimenti su cosa fosse meglio fare per alcune cose e aiuto nella ricerca di funzionalità utili ai nostri fini. Il modello usato è GeminiPro.

Prima di passare alla presentazione delle varie classi, ecco di seguito il Dependency Package Diagram, che illustra dove i vari package fanno uso di metodi esterni:

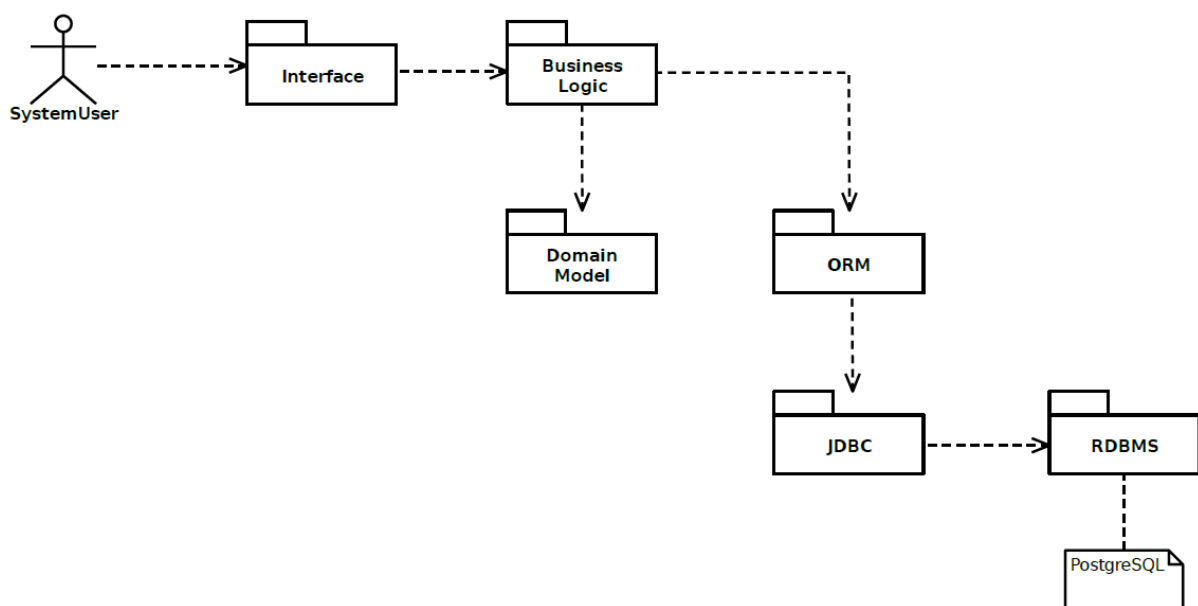


Figura 1: Package Dependencies

2 Progettazione

2.1 Use Case Diagram

L'attore **User** è il cliente utilizzatore dei servizi del software, può di base leggere le misurazioni effettuate oppure impostare una alert rule. **L'Admin** svolge l'esercizio di sorveglianza manuale del corretto funzionamento dei sensori, così come quello di amministratore generale del sistema, è l'unica entità che può bloccare un utente. Il **SystemMonitor** svolge una sorveglianza automatica (periodica) in merito al corretto funzionamento dei sensori. Sia quest'ultimo che l'admin possono aprire dei ticket in seguito al riscontro di un'anomalia, ticket che sarà preso da uno dei **Maintainer**, che interverrà sul sensore segnalato riparandolo o sostituendolo.

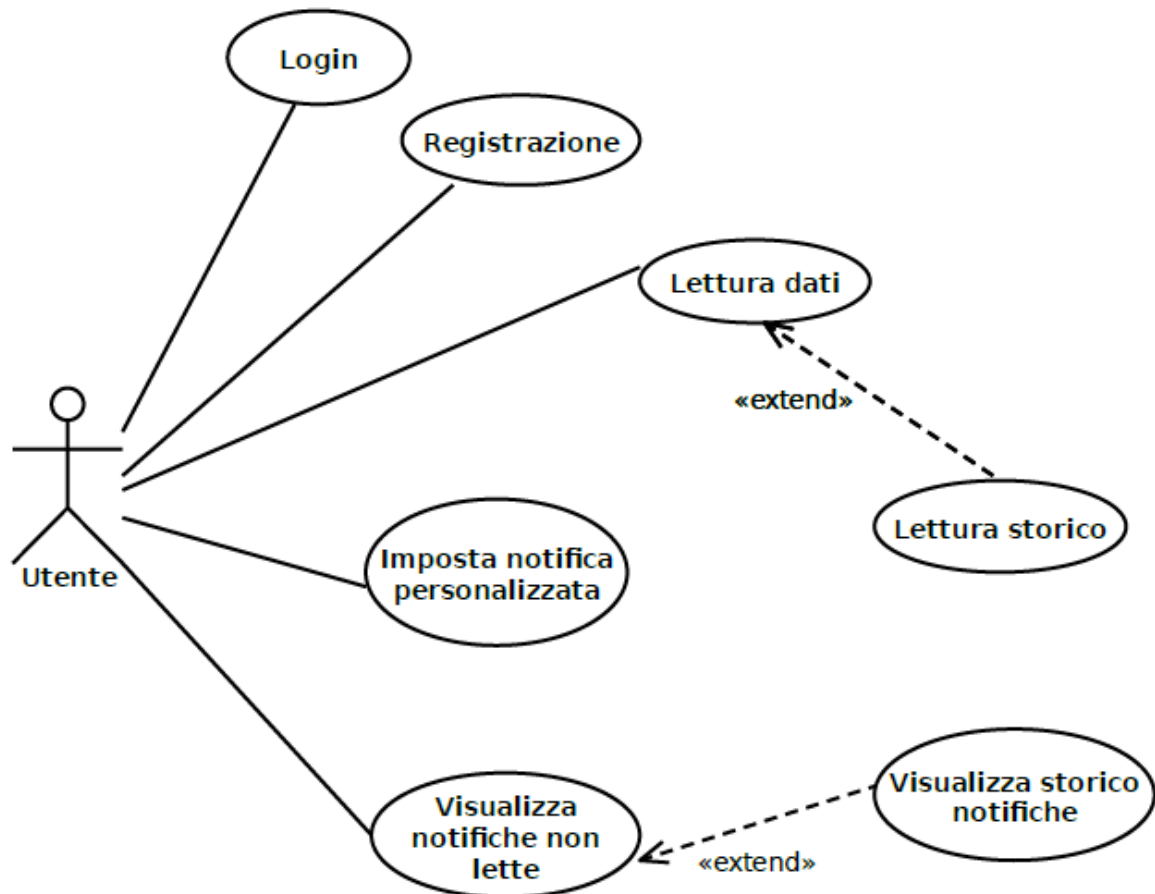


Figura 2: Use Cases User

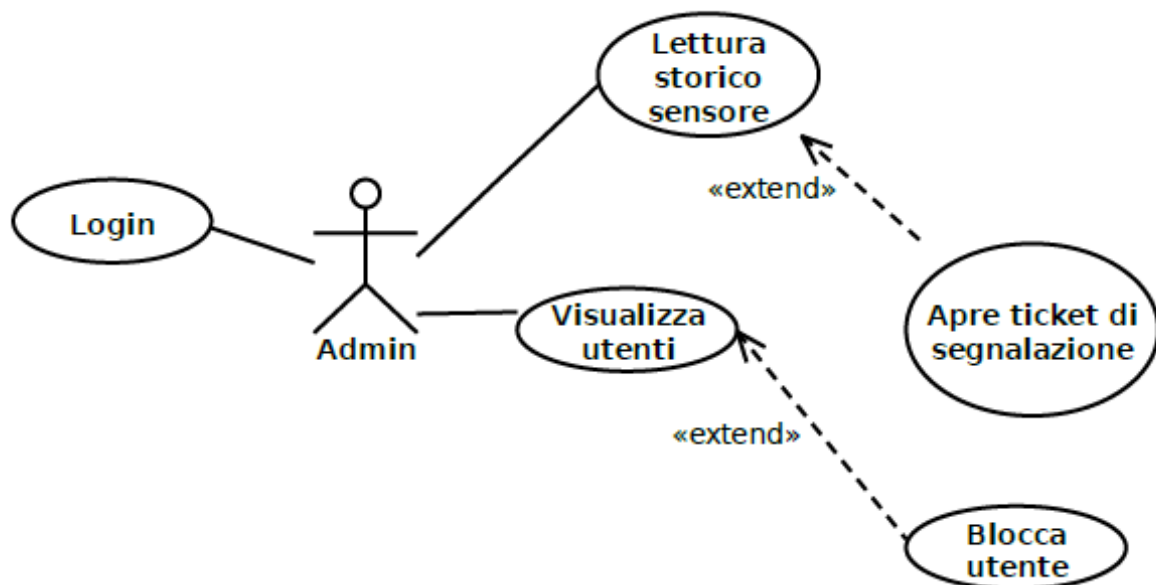


Figura 3: Use Case Admin



Figura 4: Use Case SystemMonitor

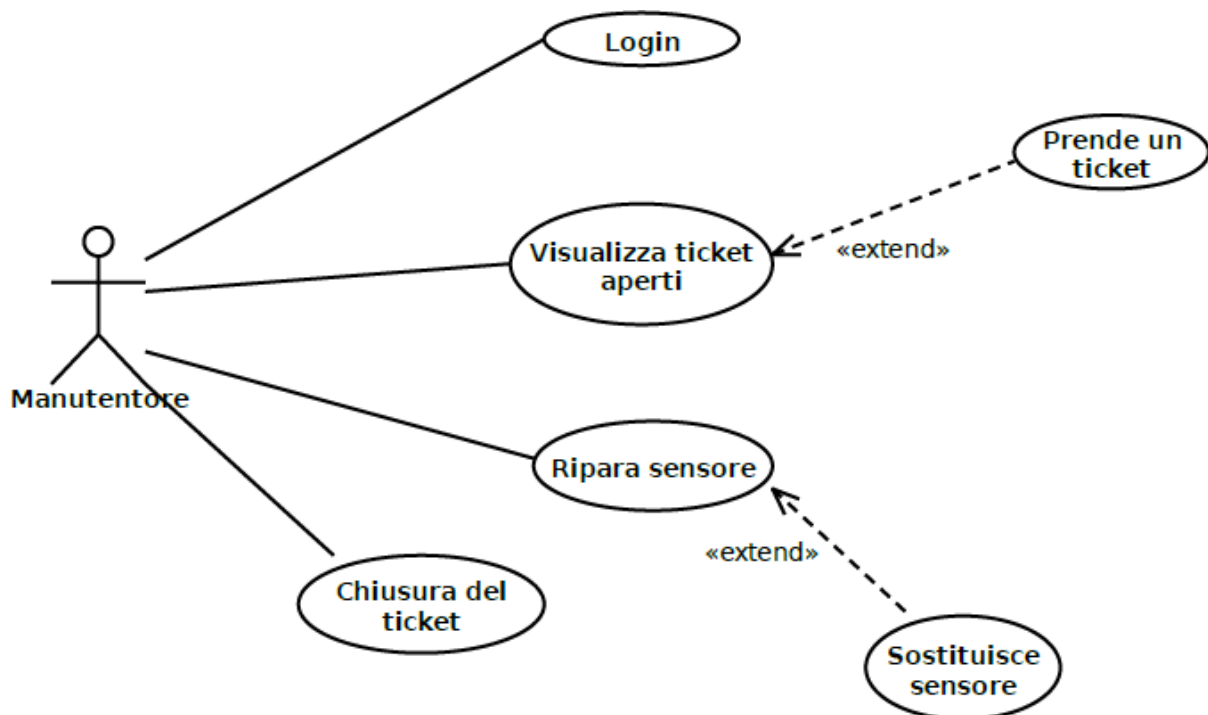


Figura 5: Use Case Maintainer

2.2 Use Case Template

Use Case #1	Accedi al sistema (Sign-in)
Brief Description	L'utente del sistema accede usando le proprie credenziali
Level	User Goal
Actors	User, Admin, Maintainer
Pre-Conditions	L'utente di sistema deve essere sulla pagina dedicata all'accesso
Basic Flow	1) L'utente di sistema seleziona la propria pagina di login 2) L'utente inserisce le proprie credenziali 3) L'utente invia le proprie credenziali 4) Il sistema verifica le credenziali 5) Il sistema autentica l'utente
Alternative Flow	4.a) Se le credenziali sono errate, l'utente non è registrato oppure è sulla pagina di login errata, il sistema restituisce un messaggio di errore e permette di ritentare
Post-Conditions	L'utente è autenticato nel sistema e ha accesso alle sue funzioni

Tabella 1: Use Case #1 (Accedi al sistema)

Use Case #2	Registrazione (Sing-Up)
Brief Description	L'utente si registra creando un nuovo account
Level	User Goal
Actors	User
Pre-Conditions	L'utente deve essere nella pagina di registrazione
Basic Flow	1) L'utente fornisce le informazioni richieste 2) L'utente conferma la registrazione 3) Il sistema verifica i dati (email ha vincolo unique) 4) Il sistema crea un nuovo account per il cliente
Alternative Flow	3.a) Se i dati non sono validi o sono già presenti in un altro account, il sistema mostra un messaggio di errore e permette di ritentare
Post-Conditions	Il sistema mostra il messaggio dell'avvenuta registrazione e riporta l'utente nella pagina di login

Tabella 2: Use Case #2 (Registrati al sistema)

Use Case #3	Apertura del ticket (Admin)
Brief Description	L'Admin apre una segnalazione su un sensore
Level	User Goal
Actors	Admin
Pre-Conditions	L'Admin deve aver fatto l'accesso e deve essere sulla pagina di "Lettura storico"
Basic Flow	1) L'Admin seleziona l'opzione "Segnala sensore" 2) L'Admin seleziona il sensore da segnalare 3) Il sistema registra il ticket sul sensore segnalato
Alternative Flow	3.a) Sul sensore segnalato è già stato aperto un ticket, quindi il sistema restituirà un messaggio di errore e non sarà aperto un ulteriore ticket.
Post-Conditions	Il ticket è stato registrato. Il sistema blocca il sensore segnalato e riporta l'Admin sulla sua dashboard, indipendentemente che l'apertura del ticket sia andata a buon fine o no.

Tabella 3: Use Case #3 (Admin: Apri ticket)

Use Case #4	Apertura del ticket (Monitor di Sistema)
Brief Description	Il Monitor di Sistema apre una segnalazione su un sensore
Level	System Goal
Actors	SystemMonitor
Pre-Conditions	Il Monitor di Sistema è "sveglio"
Basic Flow	1) Il Monitor verifica se ci sono anomalie nei sensori: verifica se l'ultima lettura dei sensori è in un range valido 2) Il sistema registra il ticket sul sensore anomalo
Alternative Flow	2.a) Il sistema non registra alcun ticket perché il Monitor non ha rilevato anomalie
Post-Conditions	Il sistema ha correttamente registrato il ticket (se è stato creato) e ha bloccato il sensore guasto

Tabella 4: Use Case #4 (SystemMonitor: Apri ticket)

Use Case #5	Prendere un ticket
Brief Description	Il manutentore prende un ticket aperto
Level	User Goal
Actors	Maintainer
Pre-Conditions	Il manutentore deve essere autenticato e nella pagina "Visualizza ticket aperti".
Basic Flow	1) Il manutentore seleziona l'opzione "Prendi ticket" 2) Il manutentore seleziona il ticket che vuole prendere 3) Il sistema registra il ticket come preso
Alternative Flow	3.a) Il sistema segnala un errore perché il ticket selezionato è già stato acquisito da un altro manutentore qualche istante prima.
Post-Conditions	Il sistema imposta nel database il ticket come "preso" e gli assegna l'id del manutentore che l'ha selezionato.

Tabella 5: Use Case #5 (Prendi un ticket)

Use Case #6	Riparazione di un sensore
Brief Description	Un manutentore che ha preso un ticket procede a riparare il sensore
Level	User Goal
Actors	Maintainer
Pre-Conditions	Il manutentore è autenticato e ha già preso un ticket
Basic Flow	1) Il manutentore ripara il sensore 1.a) Il manutentore reimposta lo stato del sensore come "attivo" 2) Il manutentore chiude il ticket
Alternative Flow	1.bis) Il manutentore sostituisce il sensore se non riesce a ripararlo 1.bis.a) Il manutentore imposta lo stato del sensore a "disattivato" 1.bis.b) Il manutentore registra un nuovo sensore dello stesso tipo di quello sostituito
Post-Conditions	Il sensore è stato riattivato oppure è stato disattivato definitivamente e sostituito

Tabella 6: Use Case #6 (Ripara sensore)

Use Case #7	Lettura dello storico delle misurazioni (User)
Brief Description	L'utente legge le misurazioni effettuate dai sensori inserendo una data di inizio e una data di fine
Level	User Goal
Actors	User
Pre-Conditions	L'utente è autenticato e sulla pagina di lettura dati
Basic Flow	1) L'utente seleziona l'opzione "Leggi storico misurazioni" 2) L'utente inserisce la data di inizio 3) L'utente inserisce la data di fine 4) Il sistema mostra tutte le misurazioni fatte da tutti i sensori in quell'arco di tempo
Alternative Flow	2/3.a) Il sistema mostra un errore se il formato della data non è valido e permette di reinserire subito tale data
Post-Conditions	Il sistema deve mostrare tutte e solo le misurazioni fatte in quell'arco di tempo

Tabella 7: Use Case #7 (User: Lettura storico misurazioni)

Use Case #8	Impostare notifiche personalizzate
Brief Description	L'utente crea una regola. Se qualsiasi sensore di quel tipo supera i limiti impostati, l'utente riceverà una notifica
Level	User Goal
Actors	User
Pre-Conditions	L'utente è autenticato e sulla pagina principale
Basic Flow	1) L'utente seleziona l'opzione "Imposta Alert Rule" 2) L'utente seleziona uno dei 3 tipi di Alert Rule: solo limite inferiore, solo limite superiore oppure entrambi 3) L'utente crea la regola impostando il tipo di sensore e il/i limite/i. Il sistema verificherà automaticamente questa regola per ogni misurazione, ad opera dei sensori di quel tipo, successiva alla creazione 4) Il sistema registra la regola nel database
Alternative Flow	3.a) Il sistema mostra un errore perché qualche valore della regola non è valido e permette di reinserirlo subito
Post-Conditions	L'utente viene riportato alla dashboard iniziale

Tabella 8: Use Case #8 (Imposta notifica personalizzata)

Use Case #9	Lettura dello storico delle notifiche (User)
Brief Description	L'utente legge tutte le notifiche risalenti ad un certo numero di giorni fa
Level	User Goal
Actors	User
Pre-Conditions	L'utente è autenticato e sulla pagina "Visualizza notifiche non lette"
Basic Flow	1) L'utente seleziona l'opzione "Leggi storico notifiche" 2) L'utente inserisce il numero di giorni antecedenti la data odierna dei quali vuole vedere le notifiche 3) Il sistema mostra tutte le notifiche appartenenti a quell'arco di tempo
Alternative Flow	2.a) Il sistema mostra un errore se l'input non è valido e permette di reinserire subito il valore
Post-Conditions	Il sistema deve mostrare tutte e solo le notifiche appartenenti a quell'arco di tempo

Tabella 9: Use Case #9 (User: Lettura storico notifiche)

2.3 Class Diagram

2.3.1 Domain Model

Contiene le entità del sistema, solo due di esse hanno al loro interno un metodo sfruttato da alcune classi della business logic. Per il resto sono solo classi di getter e setter, se non addirittura composte solo dal costruttore.

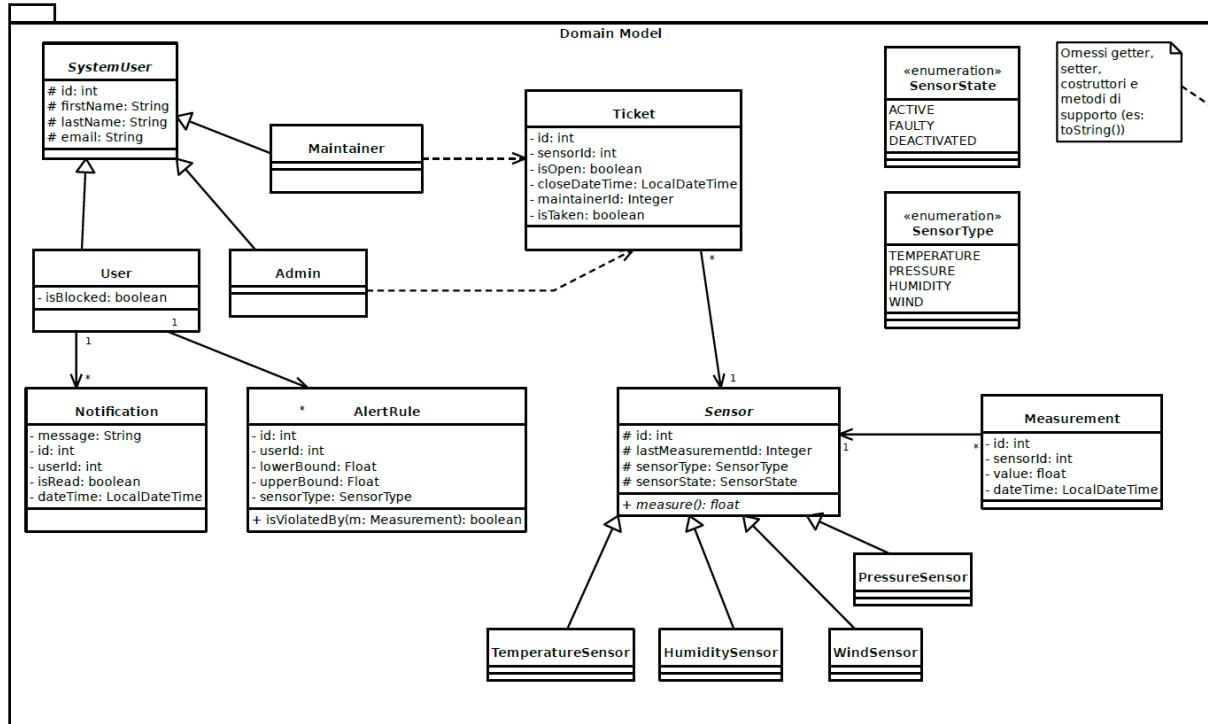


Figura 6: Class Diagram Domain Model

2.3.2 Business Logic

Si tratta di un insieme di controller che permettono agli attori di usufruire dei servizi a loro concessi. Il **SensorManager** e il **SensorMonitor** implementano il **Design Pattern Observer**. È anche presente una classe **AdminExtraController** utile per alcune funzioni di gestione del db (usate soprattutto nel testing). È anche presente un classe **DatabaseMutex** che contiene un **Semaphore** utile per la sincronizzazione dei thread **Main**, **SystemMonitor** e **SensorManager**.

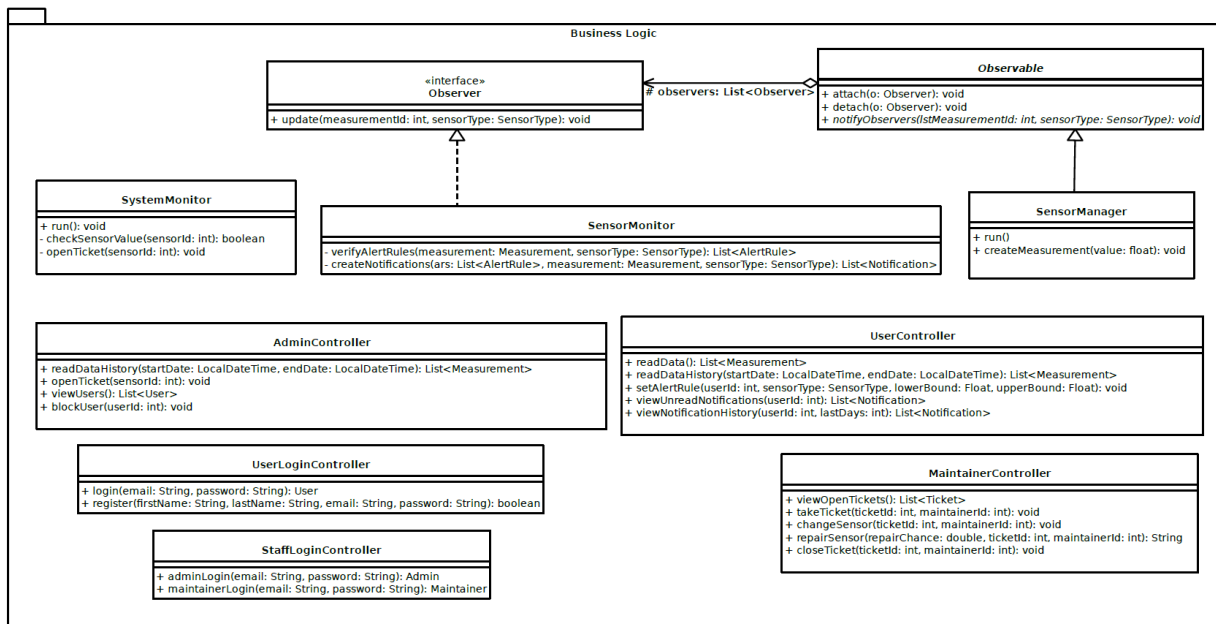


Figura 7: Class Diagram Business Logic

2.3.3 ORM

Contiene un oggetto nameClassDAO per ogni classe del domain model presente nel db di sistema. **DatabaseManager** è la classe che si occupa di erogare e chiudere la connessione al database.

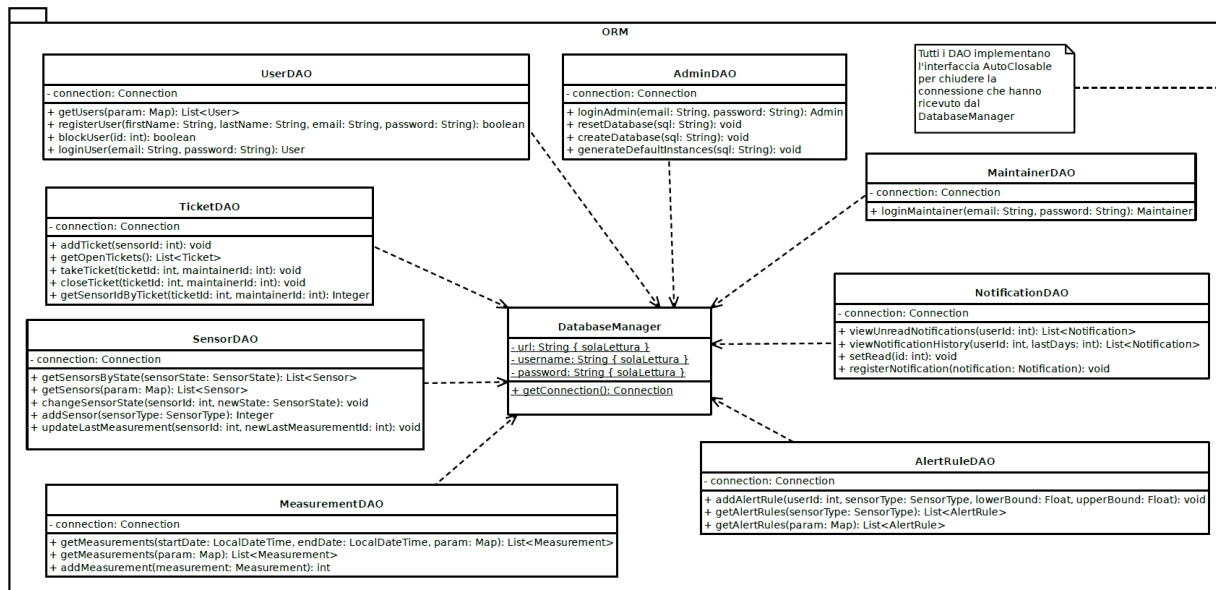


Figura 8: Class Diagram ORM

2.4 Relationship Model

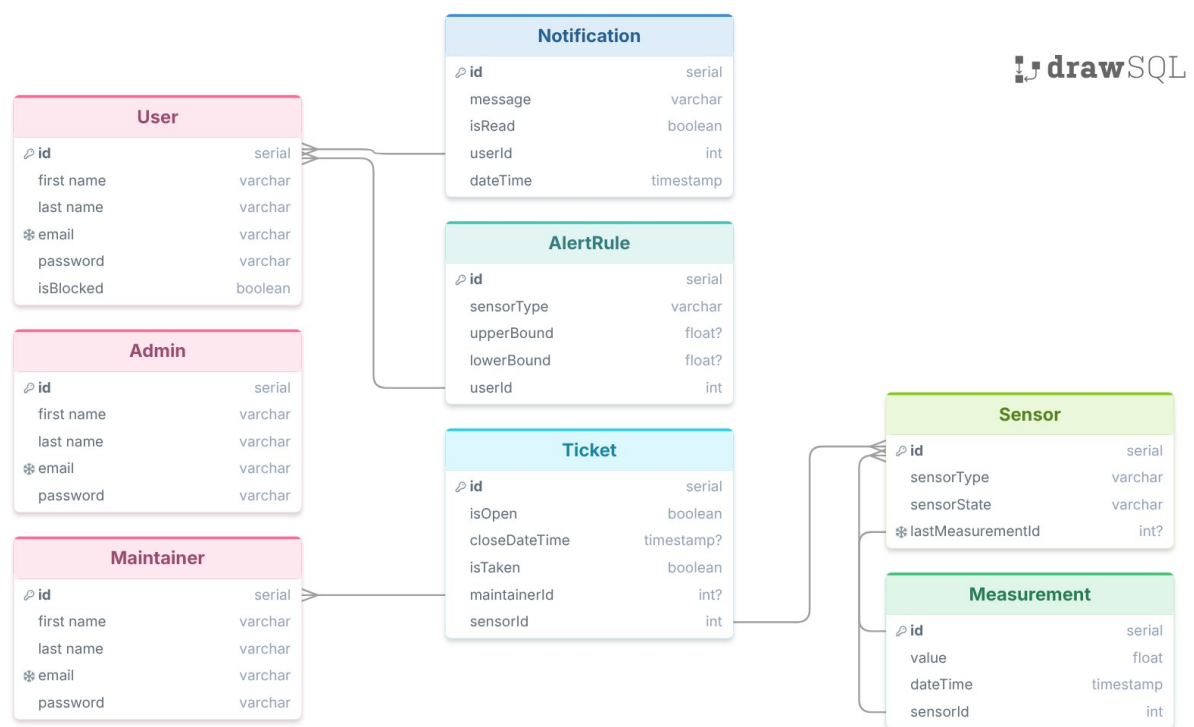


Figura 9: ER Diagram

NOTA: La tabella AlertRule è inoltre dotata dei seguenti vincoli affinché si mantenga la logica dall'AlertRule stessa:

```
CREATE TABLE IF NOT EXISTS "AlertRule"
(
    id SERIAL PRIMARY KEY,
    sensorType VARCHAR(15) NOT NULL,
    lowerBound FLOAT,
    upperBound FLOAT,
    userId INT NOT NULL,
    FOREIGN KEY (userId) REFERENCES "User" (id),
    CONSTRAINT atLeastOneBound CHECK ( lowerBound IS NOT NULL OR upperBound IS NOT NULL ),
    CONSTRAINT rightBoundOrder CHECK ( lowerBound < upperBound )
);
```

Inoltre, a causa della **relazione circolare** che c'è tra le tabelle Sensor e Measurement, si è reso necessario impostare la chiave esterna dopo la creazione di entrambe le tabelle:

```
ALTER TABLE "Measurement"
ADD CONSTRAINT fk_sensorId
FOREIGN KEY (sensorId) REFERENCES "Sensor" (id);
```

Infatti, una relazione circolare introduce il problema "È nato prima l'uovo o la gallina?", in questo caso per noi è nata prima la tabella Sensor: l'attributo lastMeasurementId potrà essere inizializzato anche in un secondo momento.

Si è reso infine necessario aggiungere anche il seguente controllo:

```
CREATE UNIQUE INDEX unique_open_ticket_per_sensor
ON "Ticket" (sensorId)
WHERE (isOpen = TRUE);
```

Per un sensore guasto infatti non possono e non devono esserci **più ticket aperti**.

2.5 Navigation Diagram

Di seguito è riportato il diagramma di navigazione dell'interfaccia a riga di comando implementata.

- **Weather station:** La home page. Si sceglie se autenticarsi come utenti o come personale.
- **Dashboard User:** Se si esegue il login come utenti si ha accesso alla lettura dei dati, all'impostazione delle alert rule personalizzate e alla lettura delle notifiche non lette. la lettura dello storico dei dati e dello storico notifiche è gestita mediante una richiesta y/n.
- **Dashboard Admin:** Può visualizzare tutti gli utenti e può leggere lo storico delle misurazioni. Sempre mediante una richiesta y/n sarà chiesto se si desidera bloccare un utente oppure aprire un ticket per un sensore.
- **Dashboard Maintainer:** Visualizza ticket aperti, quindi, con richiesta y/n, si decide se prenderne uno o meno. Può anche riparare il sensore e chiudere il ticket.

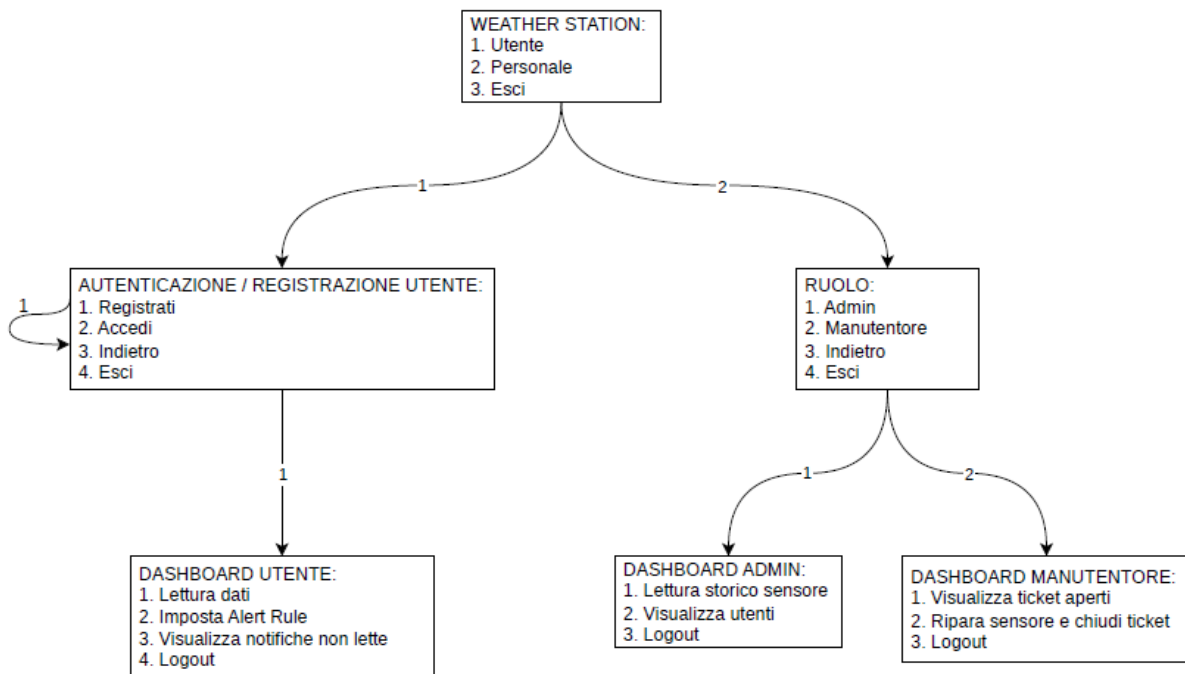


Figura 10: Navigation Diagram

3 Implementazione

3.1 Domain Model

3.1.1 SystemUser

Classe astratta che identifica un generico utilizzatore del software. Più nello specifico, è colui che può eseguire un'operazione di login mediante **email** e **password**.

- **User:** Il cliente che usufruisce del servizio come tale. Può leggere le ultime misurazioni, come può leggere lo storico delle misurazioni e può anche creare delle **Alert Rules** personalizzate.
- **Admin:** L'amministratore del sistema, nonché la figura con i privilegi più elevati. Può bloccare gli utenti e può aprire manualmente un ticket di manutenzione se, in seguito ad una lettura dei dati da lui eseguita, risultano essere presenti dei valori anomali nei vari sensori, il sensore in questione sarà settato automaticamente nello stato **SensorState.FAULTY**.

- **Maintainer:** La figura che può acquisire i ticket di manutenzione, quindi intervenire sui sensori segnalati. Se un sensore viene riparato lo resetta nello stato `SensorState.ACTIVE`, altrimenti ne aggiungerà uno nuovo impostando permanentemente a `SensorState.DEACTIVATED` quello guasto.

3.1.2 Notification

Classe che rappresenta l'oggetto che sarà inviato allo User la cui Alert Rule è appena stata violata dalle nuove misurazioni. Tra i campi, possiede il flag `isRead` che, finché sarà `false`, farà sì che la notifica in questione appaia all'utente nel contenitore delle notifiche non lette.

3.1.3 Measurement

Classe che identifica una misurazione di un sensore. Sarà quindi specificato l'id del sensore da cui è stata eseguita e il valore della misurazione. Per sapere quale sia la grandezza fisica misurata è necessario accedere alla tabella dei sensori mediante il campo `sensorId`. ATTENZIONE: le misurazioni potrebbero riportare valori sballati a causa di guasti ai sensori.

3.1.4 Ticket

Classe che identifica il ticket di segnalazione. Sarà riferito a uno specifico sensore mediante il campo `sensorId`. Possiede il flag `isTaken` e il campo `maintainerId` che, finché saranno rispettivamente false e null, faranno sì che un maintainer possa acquisire tale ticket. Una volta fatto ciò, tale flag sarà settato a true e il campo `maintainerId` sarà compilato. Il flag `isOpen` invece resterà true finché il sensore guasto non sarà o riattivato o disattivato e quindi sostituito.

3.1.5 Sensor

Classe che definisce gli strumenti di misurazione. La classe in sé è astratta ed è implementata in 4 tipi diversi di sensore: `TemperatureSensor`, `PressureSensor`, `WindSensor`, `HumiditySensor`. Nessuno di questi in realtà introduce nuovi attributi, ciò che li contraddistingue è la diversa implementazione del metodo `measure()`.

3.1.6 Enumerazioni

Sono presenti due classi che introducono due diverse enumerazioni:

- **SensorState:** assume i valori `ACTIVE`, `FAULTY`, `DEACTIVATED`, che rispettivamente rappresentano un sensore in attività regolare, un sensore che è stato contrassegnato come guasto o dall'Admin o dal SystemMonitor, un sensore che è stato permanentemente disattivato da un Maintainer.
- **SensorType:** assume i valori `TEMPERATURE`, `PRESSURE`, `WIND`, `HUMIDITY`, tali campi sono usati non soltanto all'interno dei sensori, ma anche all'interno delle AlertRule.

3.2 Business Logic

Le classi di questo package hanno lo scopo di fornire i metodi con cui le varie entità possono interagire col software. In pratica implementano gli Use Cases dei nostri Use Case Diagrams. La caratteristica trasversale di tutte le classi di questo package è che sfruttano i metodi dell'ORM per **accedere indirettamente** al db. Sarà infatti compito delle classi del package ORM implementare le query richieste.

3.2.1 AdminDatabaseController

Classe usata principalmente nel testing che ha i metodi per creare la base dati, creare le istanze di default e resettare a zero il database. I metodi della classe altro non fanno che leggere i file della cartella Resources, quindi eseguire il codice in essi contenuto. NOTA: questa è l'unica classe che fa eccezione alla descrizione generale del paragrafo precedente, infatti non sono presenti oggetti DAO.

3.2.2 AdminController

Classe che implementa i metodi `readDataHistory(...)`, con la possibilità di leggere i dati relativi o a un sensore specifico o a tutti i sensori, `viewUsers()` per vedere tutti gli utenti iscritti e `blockUser(int userId)` per bloccare un determinato utente. Infine `openTicket(int sensorId)` per aprire manualmente un ticket per un sensore.

3.2.3 MaintainerController

Troviamo i metodi `viewOpenTickets()`, `takeTicket()`, `closeTicket()` che permettono di selezionare un ticket disponibile, quindi acquisirlo se non è già stato preso. La chiusura avverrà solo in seguito alla riparazione o sostituzione di un sensore, ad opera rispettivamente delle funzioni `repairSensor(Double repairChance, int ticketId)` e `changeSensor(int ticketId)`. Le funzioni sono simulate probabilisticamente mediante la variabile `repairChance`. NOTA: in alcune di queste funzioni è necessario passare il `maintainerId`, tale compito è attribuito al Main che salva in memoria il maintainer attualmente loggato (quindi ha accesso al suo id).

3.2.4 SystemMonitor

Il SystemMonitor è un thread parallelo al Main il cui metodo `run()` (opportunamente sovrascritto dalla classe base Thread) svolge in automatico la funzione di controllo del valore registrato dai sensori mediante la funzione `checkSensorValues(int sensorId)`, qualora non risultasse ok (output false) il thread aprirà un ticket per tale sensore. Nel paragrafo successivo sarà detto qualcosa in più sulla sincronizzazione tra i thread SystemMonitor e SensorManager.

3.2.5 SensorManager – D. P. Observer

Questa classe estende la classe astratta `Observable` e implementa l'interfaccia `Runnable` e, assieme a `SensorMonitor`, va a implementare il design pattern observer del software. Il suo `run()` è l'unico metodo del software che aziona i sensori, consentendo loro di misurare tramite il metodo `measure()`. La misura viene poi opportunamente salvata nel database. Il thread invoca il metodo `notifyObservers(...)` per propagare l'Id della misurazione e il tipo di sensore al `SensorMonitor`, cosicché questo possa eseguire a sua volta il suo `update(...)` (vedi paragrafo successivo). La sincronizzazione tra questi due thread è realizzata mediante la classe `DatabaseMutex`, essa in realtà contiene solo un `Semaphore(1)` come attributo statico. I due thread prima di agire devono eseguire l'acquire di tale mutex. La logica è che nel mentre i sensori vengono controllati non possano eseguire ulteriori misurazioni, che sarebbero a loro volta errate in caso di guasto.

3.2.6 SensorMonitor – D. P. Observer

Classe che implementa l'interfaccia `Observer`. Il suo metodo `update(...)` si occupa di creare e inserire nel database le notifiche scaturite in seguito alla violazione di un bound di una `AlertRule` per una certa grandezza fisica. La verifica è effettuata mediante il metodo `verifyAlertRule(...)`.

3.2.7 UserController

Classe che definisce cosa un cliente iscritto e non bloccato può fare. Può leggere le misurazioni con `readData(...)` e `readDataHistory(...)`. Può settare una nuova alert rule personalizzata mediante `setAlertRule(...)`, un appunto che si può fare in merito a tale funzione è che è stata realizzata in modo da consentire di imporre sia un intervallo di tolleranza che solamente un limite inferiore o superiore per le misurazioni. Ad esempio, si può imporre di essere avvisati sia quando la temperatura T è solo $T < 20$, che quando è solo $T > 30$ oppure quando è l'uno o l'altro. Il cliente può inoltre leggere le notifiche non lette ed eventualmente lo storico delle notifiche con i metodi `viewUnreadNotification(...)` e `viewNotificationHistory(...)`.

3.2.8 StaffLoginController

Classe che contiene i metodi `adminLogin(String email, String password)` e `maintainerLogin(String email, String password)`, i login dello staff appunto. Si rende evidente che questo software adotta la procedura di login mediante le credenziali email e password.

3.2.9 UserLoginController

Classe che contiene i metodi `login(String email, String password)` e `register(...)`. Questi sono solo per lo User.

3.3 ORM

3.3.1 DatabaseManager e interfaccia AutoClosable

DatabaseManager è la classe che si occupa di fornire la connessione al nostro database. La logica di connessione al database è centralizzata in questa unica classe, infatti i parametri d'accesso sono tenuti solamente qui. La connessione è restituita mediante il metodo `getConnection(...)` come oggetto di tipo `Connection`. Nei vari metodiDAO la `Connection` sarà chiusa in automatico al termine del codice di ognuno di essi grazie all'implementazione dell'interfaccia `AutoClosable`. In merito a ciò può notare come ogni oggetto DAO sia inizializzato nel blocco `resources` dei vari `try` al fine di consentire il corretto funzionamento di tale macchina.

3.3.2 AdminDAO e MaintainerDAO

Contengono il metodo `login(...)` per ottenere la relativa istanza dal db. La classe `AdminDAO` contiene inoltre i metodi per creare e resettare il db, nonché inizializzarlo con le istanze di default.

3.3.3 AlertRuleDAO

Ha il metodo `addAlertRule(...)` per inserire nel db una nuova alert rule e il metodo `getAlertRule(...)` per eseguire una select che può agire sulla base o di una `Map<String, Object>` o su una variabile `SensorType` in ingresso.

3.3.4 MeasurementDAO

Similmente alla precedente `AlertRuleDAO`, c'è un metodo `addMeasurement(...)` per l'inserimento di nuove misure e un metodo `getMeasurement(...)` che può prendere in ingresso o soltanto la solita generica `Map<String, Object>` oppure, oltre a questa, anche un intervallo di date che definiranno l'intervallo temporale di cui si desidera leggere le misure effettuate dai sensori.

3.3.5 NotificationDAO

Qui è presente il metodo `registerNotification(...)` che inserisce una nuova notifica nel db. Ci sono i metodi `viewUnreadNotification(...)`, `viewNotificationHistory(int userId, int lastDays)` che filtrano la lettura delle notifiche rispettivamente per avvenuta lettura o meno e se sono risalenti a non più di `lastDays` giorni fa. In seguito alla chiamata di `viewUnreadNotification(...)` da parte di `UserController`, ci sarà anche una chiamata al metodo `setRead()` che setterà a true il campo `isRead` delle tuple relative alle notifiche appena lette.

3.3.6 SensorDAO

Ci sono due metodi per eseguire una select dei sensori: il metodo `getSensors(Map<String, Object> map)` e `getSensorByState(SensorState sensorState)`. A causa della gerarchia ereditaria, con conseguente presenza di diversi costruttori per ogni sensore, si è reso necessario uno switch-cases a seconda del tipo di sensore. Vi è poi il metodo `changeSensorState(int sensorID, SensorState newState)` che si occupa di modificare lo status di un sensore in seguito a qualche evento: l'apertura di un ticket

fa sì che un `sensorState` venga settato a `FAULTY`, la riparazione lo fa tornare nello stato `ACTIVE`, mentre la sostituzione lo rende permanentemente `DEACTIVATED`. L'inserimento di un nuovo sensore è consentito dal metodo `addSensor(...)`. Non esiste un metodo per la rimozione dei sensori: i sensori `DEACTIVATED` rimangono nel database come tali al fine non creare tuple orfane all'interno del database stesso. L'id del sensore è infatti chiave esterna di alcune tabelle. Infine abbiamo il metodo `updateLastMeasurement(...)` che modifica l'id dell'ultima misura effettuata da un sensore, tale metodo è ovviamente chiamato al momento della misura del sensore stesso.

```
try (ResultSet resultSet = statement.executeQuery()) {
    while (resultSet.next()) {
        int id = resultSet.getInt("id");
        Integer lastMeas = (Integer) resultSet.getObject("
            lastMeasurementId");
        SensorType myType = SensorType.valueOf(resultSet.
            getString("sensorType"));
        SensorState myState = SensorState.valueOf(resultSet.
            getString("sensorState"));

        Sensor sensor = switch (myType) {
            case HUMIDITY -> new HumiditySensor(id, lastMeas,
                myType, myState);
            case WIND -> new WindSensor(id, lastMeas, myType,
                myState);
            case TEMPERATURE -> new TemperatureSensor(id,
                lastMeas, myType, myState);
            case PRESSURE -> new PressureSensor(id, lastMeas,
                myType, myState);
        };

        sensors.add(sensor);
    }
} catch (SQLException e) {
    System.err.println("Errore durante la query del recupero dei
        sensori guasti: " + e.getMessage());
    e.printStackTrace();
}
```

3.3.7 TicketDAO

Per inserire nuovi ticket c'è `addTicket(...)`, in ordine poi abbiamo `getOpenTicket()`, `takeTicket(...)`, `closeTicket(...)` per le rispettive azioni. Vi è inoltre il metodo `getSensorIdByTicket(...)`, usato nel `MaintainerController`. Questo perché tramite un ticket è possibile risalire al sensore guasto, vedi schema relazionale.

3.3.8 UserDAO

Il metodo `getUsers(Map<String, Object> map)` esegue una select degli utenti col solito criterio della mappa. Ci sono poi i metodi per la registrazione e il login e il metodo `blockUser(int userId)` per settare a true il campo `isBlocked` nella tabella `User`.

4 Testing

In quest'ultima sezione sono presenti snippet di codice che mostrano come sono stati realizzati alcuni test. I test effettuati riguardano le funzioni associati agli Use Cases del software, quindi principalmente il Package della `BusinessLogic`.

4.1 Business Logic Test

Molti dei test effettuati per questo package hanno più versioni dello stesso metodo in modo da verificare ogni possibile esito verificabile, successo o fallimento che sia. I fallimenti sono stati testati per verificare il corretto funzionamento della gestione degli errori.

4.1.1 AdminControllerTest

```
@Test
void blockUser() {
    ArrayList<User> users = new ArrayList<>();
    try (UserDAO userDAO = new UserDAO()) {
        users = userDAO getUsers(Map.of("id", 2));
        assertEquals(1, users.size()); //Verifica che abbia prelevato un solo utente
    } catch (SQLException e) {
        System.err.println("Error in userDAO: " + e.getMessage());
    }
    User u1 = users.getFirst();
    adminController.blockUser(u1.getId());
    try (UserDAO userDAO = new UserDAO()) {
        users = userDAO getUsers(Map.of("id", 2));
        assertEquals(1, users.size());
    } catch (SQLException e) {
        System.err.println("Error in userDAO: " + e.getMessage());
    }
    User actualUser = users.getFirst(); //Adesso dovrebbe essere bloccato
    assertTrue(actualUser.isBlocked());

    adminController.blockUser(u1.getId()); //Provo a bloccare nuovamente lo stesso utente per vedere cosa succede
    try (UserDAO userDAO = new UserDAO()) {
        users = userDAO getUsers(Map.of("id", 2));
        assertEquals(1, users.size()); //Verifica che abbia prelevato un solo utente
    } catch (SQLException e) {
        System.err.println("Error in userDAO: " + e.getMessage());
    }
    actualUser = users.getFirst(); //Ora dovrebbe comunque essere bloccato
    assertTrue(actualUser.isBlocked());
}

@Test
void openTicket() {
    Ticket expectedTicket = new Ticket(1, 2, null, true, false, null);
    Ticket actualTicket;
    try (TicketDAO ticketDAO = new TicketDAO()) {
        adminController.openTicket(2);
        actualTicket = ticketDAO.getOpenTickets().getFirst();
        assertEquals(expectedTicket, actualTicket);
        assertThrows(SQLException.class, () -> adminController.openTicket(2)); //non puo' inserire un altro ticket ne esiste gia' uno aperto
    } catch (SQLException e) {
        System.err.println("Error in ticketDAO: " + e.getMessage());
    }
}
```

4.1.2 MaintainerControllerTest

```
@Test
void takeTicket_success() {
    try (TicketDAO ticketDAO = new TicketDAO()){
        ticketDAO.addTicket(1);
    }catch (SQLException e){
        System.err.println(e.getMessage());
    }
    assertDoesNotThrow(() -> maintainerController.takeTicket(1, 1));
}

@Test
void takeTicket_notFound() {
    try (TicketDAO ticketDAO = new TicketDAO()){
        ticketDAO.addTicket(1);
    }catch (SQLException e){
        System.err.println(e.getMessage());
    }
    assertThrows(SQLException.class ,() -> maintainerController.
        takeTicket(2, 1));
}

@Test
void takeTicket_alreadyTaken() {
    try (TicketDAO ticketDAO = new TicketDAO()){
        ticketDAO.addTicket(1);
    }catch (SQLException e){
        System.err.println(e.getMessage());
    }
    try {
        maintainerController.takeTicket(1, 1);
    } catch (SQLException e) {
        e.printStackTrace();
    }

    assertThrows(SQLException.class ,() -> maintainerController.
        takeTicket(1, 1));
}

@Test
void closeTicket_success() {
    try (TicketDAO ticketDAO = new TicketDAO()){
        ticketDAO.addTicket(1);
        maintainerController.takeTicket(1, 1);
        assertDoesNotThrow(() -> maintainerController.closeTicket(1, 1))
        ;
    }catch (SQLException e){
        System.err.println("Test viewOpenTicket exception");
    }
}

@Test
void closeTicket_alreadyClosed() {
    try (TicketDAO ticketDAO = new TicketDAO()) {
        ticketDAO.addTicket(1);
        maintainerController.takeTicket(1, 1);
        maintainerController.closeTicket(1, 1);
        assertThrows(SQLException.class, () -> maintainerController.
            takeTicket(1, 1));
    } catch (SQLException e) {

```

```

        System.err.println(e.getMessage());
    }
}

@Test
void closeTicket_notHisTicket() {
    try (TicketDAO ticketDAO = new TicketDAO()) {
        ticketDAO.addTicket(1);
        ticketDAO.takeTicket(1, 2);
        assertThrows(SQLException.class, () -> maintainerController.
            closeTicket(1, 1));
    } catch (SQLException e) {
        System.err.println(e.getMessage());
    }
}

@Test
void repairSensor() {
    double repairChance;
    boolean fixed = false, changed = false;
    try (SensorDAO sensorDAO = new SensorDAO(); TicketDAO ticketDAO =
        new TicketDAO()) {
        while (!fixed || !changed) {

            repairChance = Math.random();
            ticketDAO.addTicket(1);
            ticketDAO.takeTicket(1, 1);

            //forziamo il sensore a diventare guasto
            Map<String, Object> map = new HashMap<>();
            map.put("id", 1);
            ArrayList<Sensor> target = sensorDAO.getSensors(map);
            target.getFirst().sensorStateToFaulty();

            String result = maintainerController.repairSensor(
                repairChance, 1, 1);

            if (repairChance < 0.88) {
                target = sensorDAO.getSensors(map);
                assertEquals(SensorState.ACTIVE, target.getFirst().
                    getSensorState());
                assertThrows(SQLException.class, () -> ticketDAO.
                    closeTicket(1, 1)); //per verificare che il ticket
                    sia stato chiuso da repairSensor()
                assertEquals("sensore_riparato", result);
                fixed = true;
            } else {
                assertEquals("sensore_sostituito", result);
                changed = true;
            }
            adminDatabaseController = new AdminDatabaseController();
            try{
                adminDatabaseController.resetDatabase();
                adminDatabaseController.createDatabase();
                adminDatabaseController.defaultInstances();
            }catch (SQLException e){
                System.out.println("Errore durante il reset del database
                    ");
            }
        }
    } catch (SQLException e) {

```

```

        System.err.println("Test repairSensor exception");
        e.printStackTrace();
    }
}

```

NOTA: quest'ultimo è stato testat a tentativi vista la aleatorietà dei metodi testati

4.1.3 UserLoginControllerTest

```

    @DisplayName("User presente e credenziali esatte")
    @Test
    void login_success() {
        User user = new User(2, "Roberto", "Chiesi", "robertochiesi@test.it",
            , false);
        assertEquals(user, userLoginController.login("robertochiesi@test.it",
            , "123"));
    }

    @DisplayName("User presente, ma credenziali errate")
    @Test
    void login_fail1() {
        assertNull(userLoginController.login("robertochiesi@test.it", "abc")
        );
        assertNull(userLoginController.login("robertochiesi@email.it", "123"
        ));
    }

    @DisplayName("User non presente nel database")
    @Test
    void login_fail2() {
        assertNull(userLoginController.login("yuribartoletti@test.it", "123"
        ));
    }

    @DisplayName("User presente, ma bloccato")
    @Test
    void login_fail3() {
        assertNull(userLoginController.login("samuelezanieri@test.it", "123"
        ));
    }

    @DisplayName("Registrazione con successo")
    @Test
    void register_success() {
        assertTrue(userLoginController.register("Luke", "Skywalker", "
            lukeskywalker@test.it", "123"));
        assertNotNull(userLoginController.login("luceskywalker@test.it", "
            123"));
    }

    @DisplayName("User gia' registrato")
    @Test
    void register_fail1() {
        assertFalse(userLoginController.register("Samuele", "Skywalker", "
            samuelezanieri@test.it", "123"));
    }

    @DisplayName("Email o password troppo lunghe")
    @Test

```

```

void register_fail2() {
    StringBuilder stringBuilder = new StringBuilder(55);
    stringBuilder.repeat("a", Math.max(0, stringBuilder.capacity()));
    String email = stringBuilder.toString();
    assertFalse(userLoginController.register("Luke", "Skywalker", email,
        "123"));
    stringBuilder = new StringBuilder(260);
    stringBuilder.repeat("a", Math.max(0, stringBuilder.capacity()));
    String password = stringBuilder.toString();
    assertFalse(userLoginController.register("Luke", "Skywalker", "
        lukeskywalker@test.it", password));
}

```

La logica dei test della classe StaffLoginController è fondamentalmente la solita, dunque è stata omessa in questa relazione

4.1.4 UserControllerTest

```

@Test
void setAlertRule() {
    userController.setAlertRule(1, SensorType.TEMPERATURE, 11f, 51f);
    userController.setAlertRule(2, SensorType.WIND, null, 89f);
    userController.setAlertRule(3, SensorType.HUMIDITY, 36f, null);

    ArrayList<AlertRule> comparing = new ArrayList<>();
    ArrayList<AlertRule> results = new ArrayList<>();
    comparing.add(new AlertRule(7, 1, 11f, 51f, SensorType.TEMPERATURE))
        ;
    comparing.add(new AlertRule(8, 2, null, 89f, SensorType.WIND));
    comparing.add(new AlertRule(9, 3, 36f, null, SensorType.HUMIDITY));
    try (AlertRuleDAO alertRuleDAO = new AlertRuleDAO()){
        Map<String, Object> map = new HashMap<>();
        map.put("id", 7);
        results.add(alertRuleDAO.getAlertRules(map).getFirst());
        map.clear();
        map.put("id", 8);
        results.add(alertRuleDAO.getAlertRules(map).getFirst());
        map.clear();
        map.put("id", 9);
        results.add(alertRuleDAO.getAlertRules(map).getFirst());

    }catch (SQLException e){
        System.err.println("Error in setAlertRule Test: "+e.getMessage()
            );
    }

    assertIterableEquals(comparing, results);
}

```

4.2 Domain Model

Di questo package è stato realizzato sollo il testing per il metodo `isViolatedBy(...)` della classe `AlertRule`, che verifica se una misurazione ha violato i suoi bound.

4.2.1 AlertRuleTest

```

@Test
void isViolatedBy_lowerBoundViolated() {

```

```

        Measurement measurement = new Measurement(0, 0, 14f, null);
        assertTrue(alertRule.isViolatedBy(measurement));
    }

    @Test
    void isViolatedBy_upperBoundViolated() {
        Measurement measurement = new Measurement(0, 0, 51f, null);
        assertTrue(alertRule.isViolatedBy(measurement));
    }

    @Test
    void isViolatedBy_valueInRange() {
        Measurement measurement = new Measurement(0, 0, 35f, null);
        assertFalse(alertRule.isViolatedBy(measurement));
    }

    @Test
    void isViolatedBy_valueEqualsLowerBound() {
        Measurement measurement = new Measurement(0, 0, 15f, null);
        assertFalse(alertRule.isViolatedBy(measurement));
    }

    @Test
    void isViolatedBy_valueEqualsUpperBound() {
        Measurement measurement = new Measurement(0, 0, 50f, null);
        assertFalse(alertRule.isViolatedBy(measurement));
    }
}

```